

Programación Lógica

Obligatorio 2 – 2026

Facultad de Ingeniería

Instituto de Computación

El objetivo de este obligatorio es implementar un juego de cartas en Prolog y una estrategia inteligente para que lo juegue.

Nota previa - IMPORTANTE

Se debe cumplir íntegramente el "Reglamento del Instituto de Computación ante Instancias de No Individualidad en los Laboratorios". En particular está prohibido utilizar documentación de otros grupos o de otros años, de cualquier índole, o hacer público código a través de cualquier medio (EVA, correo, papeles sobre la mesa, etc.).

Las actividades del curso están pensadas para que cada estudiante genere su propio código. No está permitido el uso de herramientas de IA generativa. Estos lineamientos están basados en la [Guía de uso crítico y ético de IA en las UC de Fing](#)

Introducción

El Gin Rummy es un juego de cartas para dos jugadores con un [mazo francés](#) de 52 cartas. El objetivo es formar combinaciones válidas (**melds**) y dejar el menor valor posible en las cartas que no entran en ningún meld (**deadwood**). Una ronda puede terminar cuando un jugador **corta**; en esta versión simplificada, el ganador de la ronda se determina comparando el deadwood final (el jugador con el menor valor gana).

GinRummyLog es una variante de laboratorio: una sola ronda, dos jugadores, reglas de meld y deadwood acotadas como se detalla abajo. No se exige implementar toda la puntuación ni las reglas avanzadas del Gin Rummy clásico.

Se debe implementar en Prolog un módulo que permita decidir, en cada turno:

- si robar del **mazo** o del **tope del descarte**;
- **qué carta descartar**;
- si **cortar** la ronda o **continuar**.

Esas decisiones dependen de la **estrategia** (**random**, **greedy**, **pro**) y de la información disponible: mano propia, tope del descarte y cartas vistas (definidas más abajo).

Reglas del GinRummyLog

Inicio de la ronda

- Cada jugador recibe **10** cartas.
- El resto forma el **mazo**.
- Se voltea **una** carta para iniciar la **pila de descarte**; el tope es la carta visible.

Desarrollo de un turno

En cada turno, el jugador en juego:

1. **Roba** exactamente una carta: o bien la del tope del descarte, o bien la superior del mazo.
2. **Descarta** una carta de su mano (tras el robo tiene 11 cartas; tras descartar vuelve a 10).
3. **Decide** si **corta** la ronda o **continúa**. Si corta, la ronda termina. Importante: para poder cortar el valor del deadwood tiene que ser menor o igual a 10.

Melds

Combinaciones válidas:

- **Set**: tres o cuatro cartas del **mismo valor** y **palos distintos**.
- **Run**: tres o más cartas **consecutivas** en valor y del **mismo palo**.

Deadwood

Las cartas de la mano que **no** se asignan a ningún meld en una descomposición dada forman el **deadwood** relativo a esa descomposición. Su **valor** es la suma de:

- As: **1**
- 2–10: su número
- J, Q, K: **10** cada una

Cortar

Un jugador solo puede elegir **cortar** si el **deadwood** de su mano es **menor o igual a 10**.

Importante: una estrategia puede **seguir jugando** aunque ya pudiera cortar legalmente (por ejemplo para mejorar la mano).

Modelo de información: cartas vistas

Cartas vistas son las cartas que están en el descarte y no son el tope. Estas son cartas que salieron y no van a volver a salir y, por lo tanto, es imposible que el jugador las obtenga. Algo importante sobre esto es que no estamos modelando el caso donde el rival **robó del descarte** una carta que antes era el tope, esa carta pasa a su mano y **deja de estar visible**; en este caso esa carta **no** figura como “carta vista”. Dicho de otra forma, no se modela el conocimiento de las cartas que sabemos que el rival tiene.

Representación

- **Carta**: $c(\text{Valor}, \text{Palo})$.
 - $\text{Palo} \in \{c, d, t, p\}$ (corazón, diamante, trébol, pica)
 - $\text{Valor} \in \{a, 2, 3, 4, 5, 6, 7, 8, 9, 10, j, q, k\}$.
- **Mano**: lista de $c(\text{Valor}, \text{Palo})$ (el orden en la lista no importa).

Predicados a implementar

Todos los predicados deben **terminar en tiempo razonable** y **no** producir ramas infinitas al pedir más soluciones. Se deben implementar los siguientes predicados:

- **is_meld(+Cartas)**: Se cumple si y solo si Cartas es un **set** o un **run** válido según las reglas anteriores (el orden de las cartas en la lista **no** importa).
 - *Ejemplos*:
 - $\text{is_meld}([c(2,c), c(a,c), c(3,c)]) \rightarrow \text{true (run)}$.

- $\text{is_meld}([c(a,c), c(a,d), c(a,t)]) \rightarrow \text{true}$ (set de tres).
 - $\text{is_meld}([c(a,d), c(3,t), c(4,t)]) \rightarrow \text{false}$.
- **valor_deadwood(+Cartas, ?Valor)**: Valor es la suma de valores de puntos de las cartas en Cartas según la tabla de deadwood.
 - *Ejemplos*:
 - $\text{valor_deadwood}([c(2,c), c(a,c), c(3,c)], X) \rightarrow X = 6$.
 - $\text{valor_deadwood}([c(q,c), c(j,c), c(3,c)], X) \rightarrow X = 23$.
 - $\text{valor_deadwood}([c(q,c), c(j,c), c(3,c)], 2) \rightarrow \text{falso}$.
- **get_melds(+Mano, ?Melds, ?Sobrantes)**: Melds es una lista de listas; cada sublista es un meld válido. Este predicado puede ser **no determinista**: una misma mano puede admitir varias descomposiciones. En ningún caso importa el orden de las listas. Observar que este predicado genera todas las descomposiciones, la que minimiza el valor del deadwood y las que no.
 - *Ejemplos*:
 - $\text{get_melds}([c(a,t), c(2,t), c(3,t), c(9,p), c(j,t)], X, S)$
 $\rightarrow X = [[c(a,t), c(2,t), c(3,t)], S = [c(9,p), c(j,t)]$
 (esta es una de las soluciones que devuelve)
 - $\text{get_melds}([c(a,t), c(2,t), c(3,t), c(9,p), c(j,t)], [], S)$
 $\rightarrow S = [c(a,t), c(2,t), c(3,t), c(9,p), c(j,t), c(9,p), c(j,t)]$
- **best_melds(+Mano, ?MejorMelds, ?Sobrante, ?Valor)**: De todas las particiones que pueden obtenerse via get_melds nos quedamos con la que minimiza el valor del deadwood. Si hay varias particiones que minimizan este valor puede devolverse cualquiera de ellas. Tiene la restricción de que MejorMelds no puede tener más de 10 cartas.
- **robar(+Mano, +Descarte, +CartasVistas, +Estrategia, ?Lugar)**: Este predicado decide de donde vamos a robar la carta. Descarte: carta en el tope del descarte (término $c/2$). CartasVistas: lista de cartas vistas según la definición de arriba. Estrategia $\in \{\text{random, greedy, pro}\}$. Lugar $\in \{\text{mazo, descarte}\}$.
- **descartar(+OldMano, +CartasVistas, +Estrategia, ?NewMano, ?NewDescarte)**: Este predicado decide qué carta vamos a descartar. En el momento del descarte, OldMano tiene **11** cartas (ya robó). Debe cumplirse que: NewDescarte es una carta que aparece en OldMano; NewMano es OldMano sin **una** ocurrencia de esa carta; por tanto, NewMano tiene 10 cartas.
- **cerrar(+Mano, +CartasVistas, +Estrategia, ?Decision)**: Este predicado decide si continuamos jugando o si cortamos. Mano tiene **10** cartas (ya descartó en ese turno). Decision $\in \{\text{continuar, cortar}\}$. Si Decision = cortar, debe cumplirse la **regla de legalidad**: $\text{deadwood} \leq 10$.

Estrategias Deben implementarse al menos tres estrategias diferentes:

- **random**: Sin heurística: las elecciones entre opciones válidas se realizan al azar (p. ej. con los predicados de aleatoriedad de SWI-Prolog). Debe respetarse la legalidad al unificar cortar ($\text{deadwood} \leq 10$).
- **greedy**: En cada paso se toma una decisión localmente razonable, orientada a reducir o mantener bajo el deadwood según best_melds, sin planificar a largo plazo. En esta estrategia no es necesario que utilicen la información de las cartas vistas. Ejemplo: al decidir el robo, comparar el deadwood óptimo de la mano actual frente a la mano que resultaría de tomar el tope del descarte.
- **pro**: Estrategia libre: ustedes diseñan el mejor jugador que puedan. La idea es que traten hacer un jugador que juegue mejor que la estrategia greedy.

Entregable El archivo .zip a entregar debe contener:

1. Código fuente: ginrummylog.pl con los predicados obligatorios y otros archivos que utilicen para implementar su solución.
2. Informe en formato pdf detallando las decisiones de diseño. El informe debe incluir al menos:
 - **Introducción:** problema, enfoque y principales dificultades y cómo las resolvieron.
 - **Estrategias:** descripción de cada estrategia (random, greedy y pro) explicando las decisiones de diseño aplicadas en cada una. Incluir ejemplos con mediciones de time/1 sobre consultas representativas de los predicados. Hacer énfasis en cómo implementaron la estrategia pro y cómo pensaron la solución. Si probaron distintos enfoques que no funcionaron también tiene valor que lo comenten.
 - **Evaluación experimental:** para la evaluación tienen que comparar la estrategias: a) greedy vs random y b) pro vs greedy. Ejecutar 25 partidas para cada caso (cada partida con una seed distinta). Se tiene que reportar en una tabla:
 - i. Cuántas iteraciones se tardó en terminar la partida.
 - ii. Valor final del deadwood de cada participante.
 - iii. Quién ganó
 - iv. Seed utilizada
 - Lo anterior es lo mínimo necesario a entregar en el informe, sientan la libertad de hacer más experimentos.

Insumos:

Cuentan con un archivo server.pl para simular por terminal una partida.

Para ejecutarlo hacen "swipl server.pl" en su terminal y dentro del intérprete ejecutan "main."

Importante: todo lo que ingresen dentro del intérprete tiene que terminar con un "."

También cuentan con un archivo test.pl, para correr esto hacen "swipl test.pl" y dentro del intérprete ejecutan "run_tests."

Forma de entrega La entrega se realizará a través del espacio eva del curso. Se debe entregar un solo archivo 'grupo##.zip', donde ## es el número del grupo que realiza la entrega.